Aula 11 – Análise Sintática Ascendente: Prática sobre *Parser* para Expressões Aritméticas utilizando SLR(1)

Prof. Rogério Aparecido Gonçalves *Universidade Tecnológica Federal do Paraná (UTFPR)*

Nesta aula prática o objetivo é projetar e implementar um autômato com pilha, que funcione como um Analisador Sintático *(parser)* para expressões aritméticas. É esperado que o programa, por linha de comando, receba como entrada um arquivo com um expressão ou atribuição e gere como saída se são válidas ou não nas regras gramaticais. A parte da gramática relacionada a expressões aritméticas são apresentados, bem como as instruções para o desenvolvimento. Para o projeto será desenvolvido uma versão no JFLAP utilizando a gramática esta etapa servirá para a produção de figuras, tabelas para o relatório, e a implementação em Linguagem de Programação de um programa que execute o comportamento do Autômato.

Sumário

1 Análise Sintática **1**
1.1 Análise SLR(1) 2
1.2 Construção do Autômato de Itens 2

2 Desenvolvimento da Atividade **4**
2.1 Linguagem C- 4
2.2 Implementação JFLAP 5
2.3 Implementação em Código 8

3 Relatório **10**

4 Referências **11**
4.1 *Word Cloud* 11
Referências 11

1 Análise Sintática

A **Análise Sintática** determina a sintaxe ou estrutura de um programa. A sintaxe de uma *LP* é normalmente dada pelas **regras gramaticais** de uma **Gramática Livre de Contexto** (GLC). Assim como a estrutura léxica é determinada por expressões regulares, uma GLC utiliza convenções para nomes e operações muito similares às usadas por expressões regulares. A diferença é que as regras de uma gramática livre de contexto podem ser **recursivas**.

Existem duas categorias gerais de algoritmos para análise sintática:*Análise Sintática Ascendente* e *Análise Sintática Descendente*. O que determina como é feita a *análise sintática* e como *árvore sintática* correspondente é construída.

No processo de Análise Sintática o **Analisador Sintático** ou **parser** é responsável por:

- determinar a estrutura sintática de um programa a partir das **marcas** (tokens) produzidos pelo processo de **varredura (scanner, Analisador Léxico)**
- construir, explícita ou implicitamente, uma árvore sintática que representa essa estrutura.
- ao final teremos uma Árvore Sintática construída com as derivações das **regras sintáticas**.
- O analisador sintático é o consumidor da sequências de marcas produzidas pelo processo de varredura e gera a árvore sintática com a derivação das regras.

Um Analisador Sintático Ascendente usa uma pilha explícita para efetuar a análise, de maneira similar a um analisador descendente não-recursivo.

A pilha conterá tanto marcas como não-terminais e também informações adicionais de estados. A pilha está vazia no início do processo de análise e conterá o símbolo inicial ao final de uma análise bem sucedida.

Um Analisador Ascendente tem duas ações possíveis, além de `aceita`:

1. **Shift** (carrega): um terminal do início da entrada é carregado para o topo da pilha.
2. **Reduce** (reduz): uma cadeia α do topo da pilha é reduzida para um não-terminal A, se existir uma regra na BNF $A \rightarrow \alpha$.

Assim, um analisador ascendente também pode ser denominado de analisador *shift-reduce* (carrega-reduz). A ideia é a generalização das regras, uma cadeia é reduzida de acordo com uma dada regra para o símbolo inicial da regra.

1.1 Análise SLR(1)

A Análise Sintática `LR(1)` simples ou `SLR(1)` utiliza um `DFA` de conjuntos de itens `LR(0)`, aumentando o poder de análise `LR(0)`, pois utiliza a marca seguinte na cadeia de entrada para dirigir suas ações:

1. Consuta a marca de entrada antes de carregar para garantir que exista uma transição apropriada no `DFA`.
2. Utiliza o conjunto `FOLLOW` (Seguinte) de um não-terminal para decidir se uma redução deve ser feita.

1.2 Construção do Autômato de Itens

Considere o exemplo:

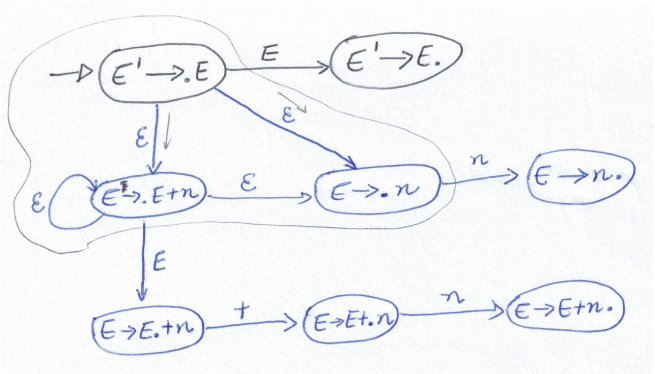
Gramática:	**Gramática Aumentada:**
	$E' \rightarrow E$
$E \rightarrow E + n \vert n$	$E \rightarrow E + n \vert n$

Os itens LR(0) de uma `GLC` são as escolhas de produções com uma posição identificada em seu lado direito. Essa posição é indicada por um ponto (.).

Itens:

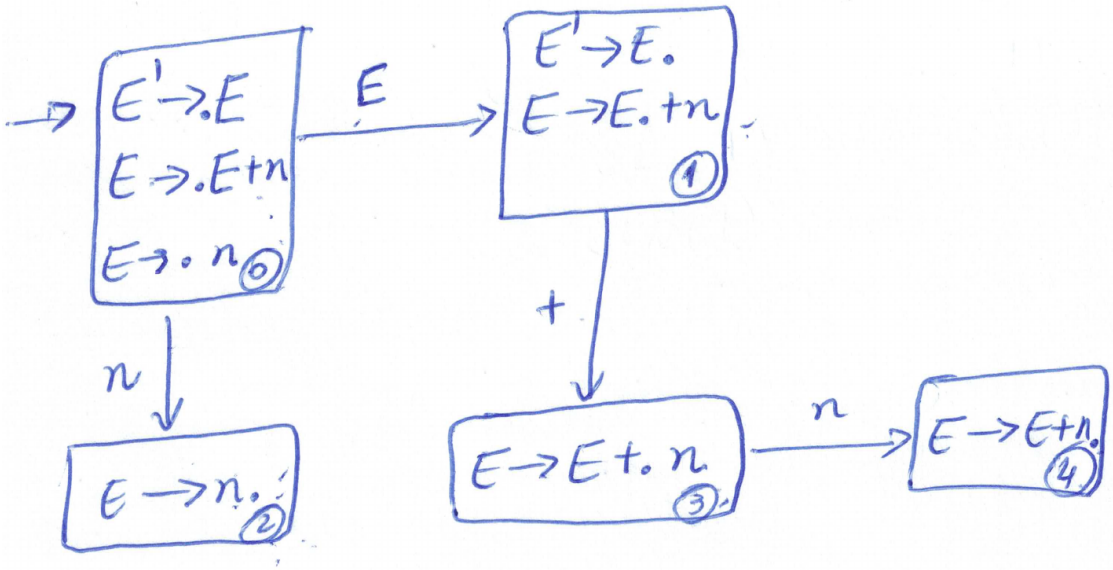
$E' \to .E$
$E' \to E.$
$E \to .E + n$
$E \to E. + n$
$E \to E + .n$
$E \to E + n.$
$E \to .n$
$E \to n.$

Autômato



Subconjuntos:

1. **Estado Inicial:** onde consigo chegar com ϵ: $\{E' \to .E, E' \to .E + n, E' \to .n\}$. Existe uma transição em E de $E' \to .E$ para $E' \to E.$ e $E' \to .E + n$ para $E' \to E. + n$. Portanto, existe uma transição do Estado inicial para o conjunto $\{E' \to E., E' \to E. + n\}$



Para a implementação, as ações de `shift` e `reduce` produzem uma tabela chamada **"action-goto"**.

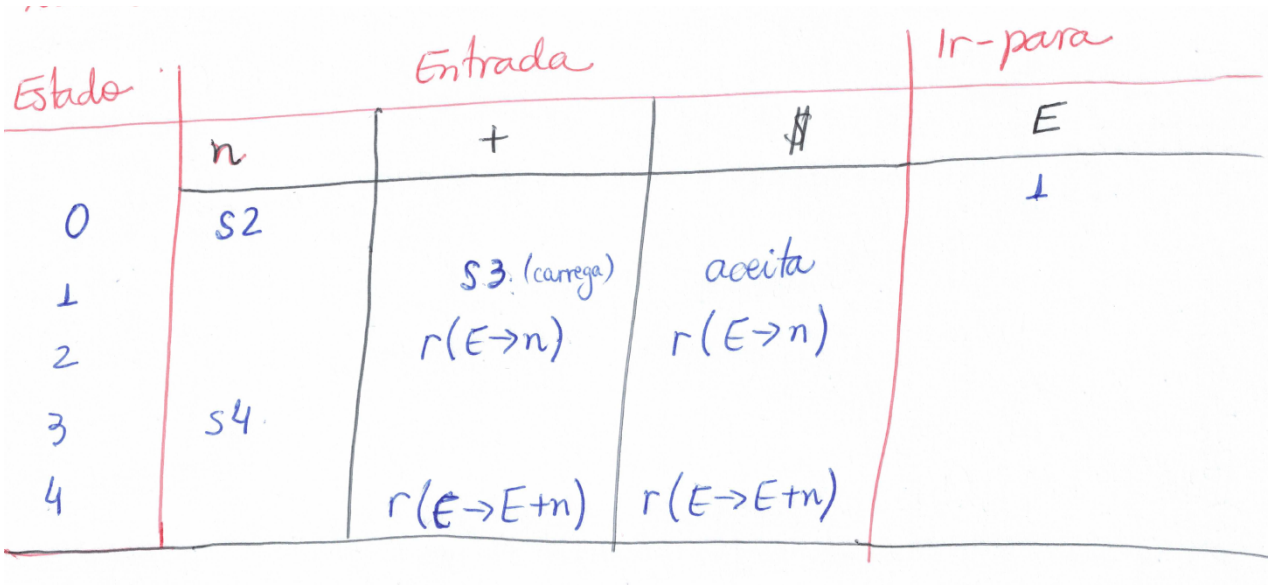


Tabela de transição `action-goto`

- 4 tipos de ações:
  - **s**n: shift para o estado n;
  - **g**n: vá para o estado n;
  - **r**k: reduza pela regra k;
  - **a**: accept;
  - : error (entrada em branco).
- As arestas do DFA são as ações *shift* e *goto*
- No exemplo anterior, cada linha indica o estado destino.

Um exemplo de execução para a cadeia $n + n + n$ é apresentado na Figura 1.

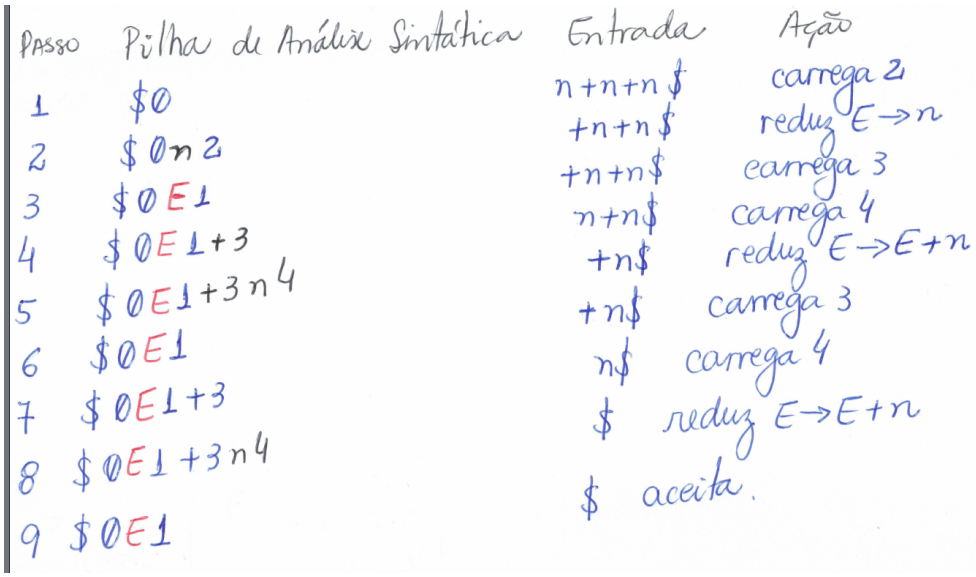


Figura 1: Exemplo para a cadeia n+n+n

O livro *JFLAP: An Interactive Formal Languages and Automata Package* (Rodger 2006), em seu *Capítulo 8: LL and SLR Parsing*, seção *8.3. SLR(1) Parsing* apresenta a Análise SRL(1) no `JFLAP`.

2 Desenvolvimento da Atividade

O desenvolvimento dessa atividade de implementação do algoritmo de *parsing* de expressões aritméticas e atribuições, pode ser feita de duas formas: no JFLAP ou utilizado uma linguagem de programação de sua preferência.

2.1 Linguagem C-

A versão simplificada, C- é apresentada no *Apêndice A* do livro texto utilizado na disciplina de Compiladores (LOUDEN 2004). Para essa atividade iremos utilizar somente as regras relacionadas a expressões aritméticas na implementação de um *Analisador Sintático (parser)* utilizando *Autômato com Pilha* (Menezes 2011) (Sipser 2007). As regras sintáticas relacionadas à expressões aritméticas são apresentadas no Código 1.

```
expression ::= var = expression | simple-expression
var ::= ID | ID [ expression ]
simple-expression ::= additive-expression relop additive-expression | additive-expression
relop ::= <= | < | > | >= | == | !=
additive-expression ::= additive-expression addop term | term
addop ::= + | -
term ::= term mulop factor | factor
mulop ::= * | /
factor ::= ( expression ) | var | call | NUMBER
```

Excluindo os operadores relacionais, deixando apenas os operadores aritméticos e atribuição com variáveis escalares, temos o Código 2.

```
expression ::= var = expression | additive-expression
var ::= ID
additive-expression ::= additive-expression addop term | term
addop ::= + | -
term ::= term mulop factor | factor
mulop ::= * | /
factor ::= ( expression ) | var | NUMBER
```

2.2 Implementação JFLAP

A implementação no JFLAP inicia com a edição da gramática na ferramenta e gerando-se o *Autômato de Itens* e a *Tabela de Transição*.

A edição da gramática pode ser feita a partir do menu inicial do JFLAP apresentado na Figura 2. Quando clicamos em *Grammar* a tela de edição presenta na Figura 3 será apresentada.

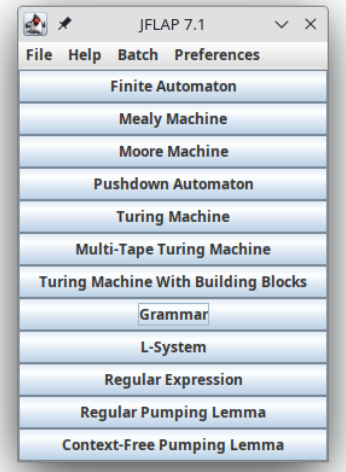


Figura 2: Editando a Gramática no JFLAP

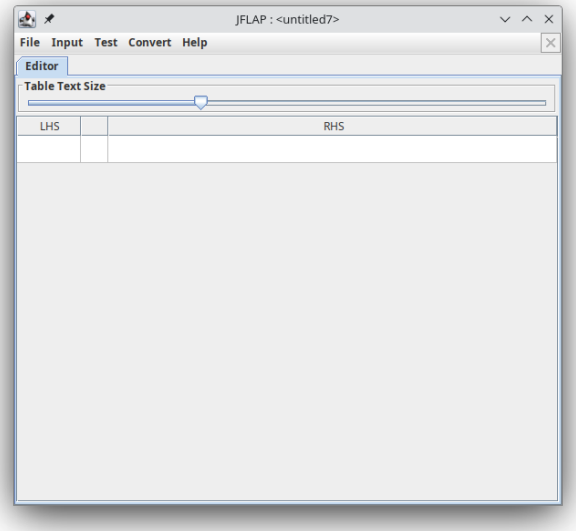


Figura 3: Editando a Gramática no JFLAP

Após digitar as regras conforme é a Figura 4. Clicando no menu **Input** é possível escolher entre as opções de *Parser* disponíveis.

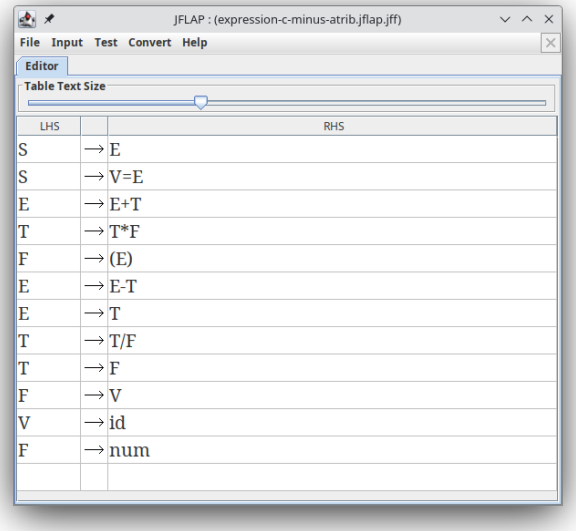


Figura 4: Editando a Gramática no JFLAP

Utilizaremos a opção **Build SLR(1) Parse Table** para implementar a *Análise SLR(1)* conforme descrita na Seção 1.1. Clicando-se no botão **Do All** o `JFLAP` irá gerar os conjuntos `FIRST` (primeiro) e `FOLLOW` (seguinte), gerará também o *Autômato de Itens* e a *Tabela de Transição*. Esses artefatos são apresentados na Figura 5.

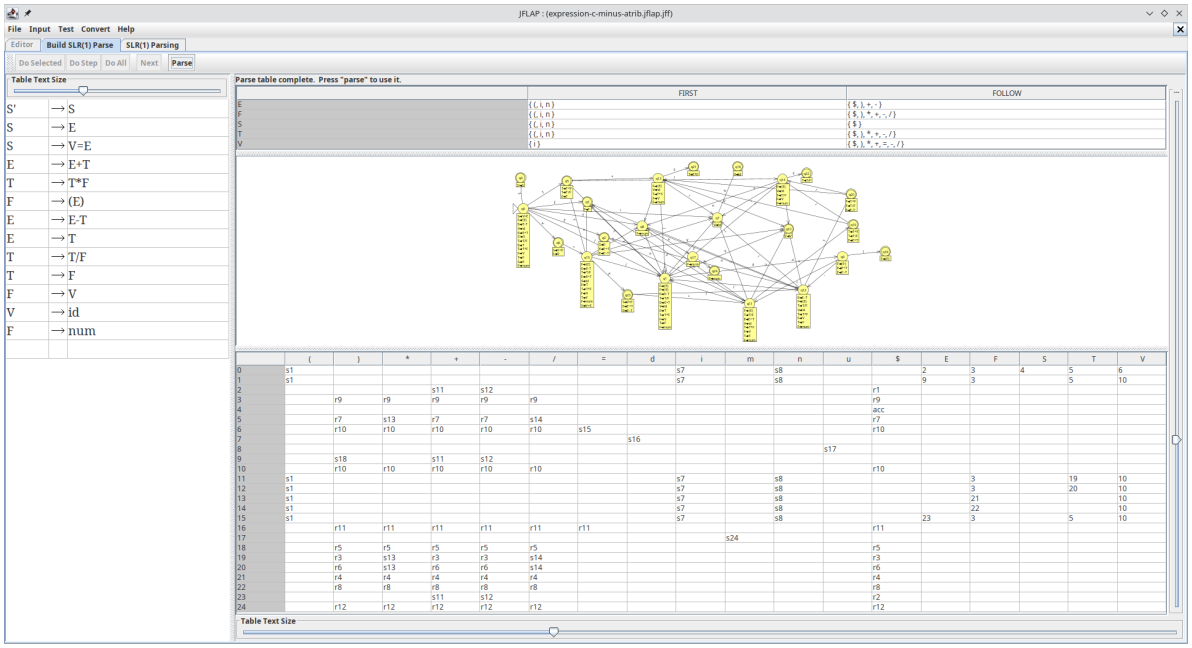


Figura 5: Autômato de Itens no JFLAP

Clicando em **Parse**, uma tela como a da Figura 6 será apresentada. Nesta tela é possível fazermos o *parsing* de expressões como $id = (id * num) + id$, clicando em *Start* e depois executando

passo a passo com *Step*.

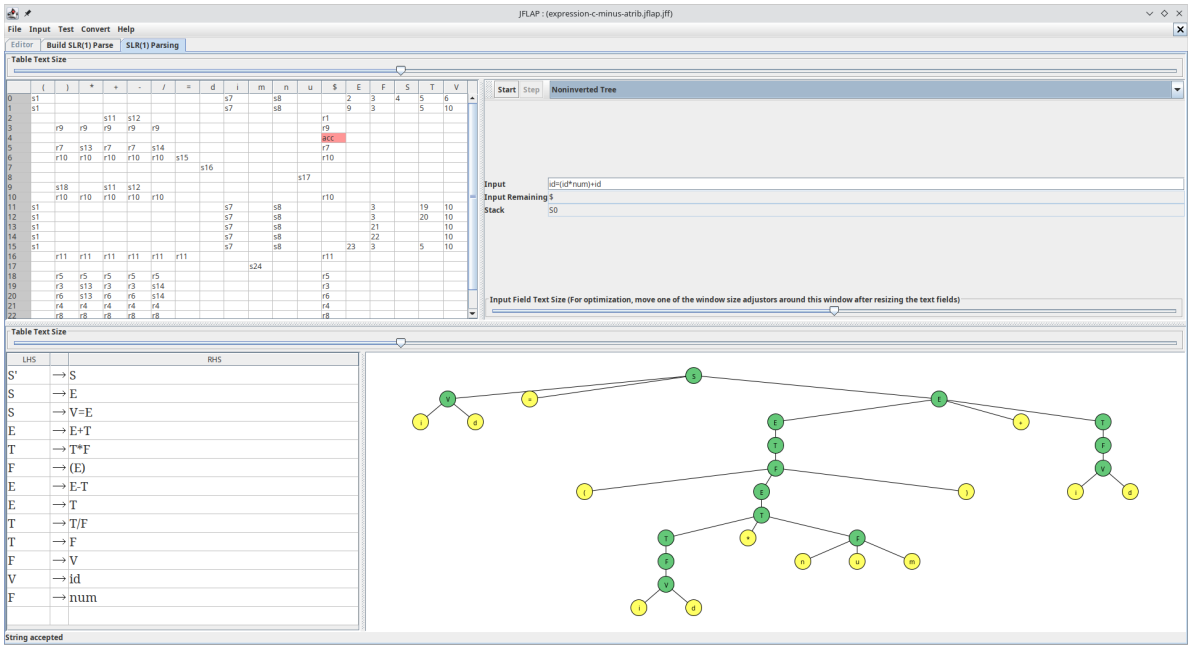


Figura 6: Parsing com JFLAP

Utilize o `JFLAP` para gerar o Autômato e produzir figuras para o relatório. É interessante também para a mesma gramática verificar na opção de "Força Bruta" do `JFLAP` quantos nós são gerados para expressões do tipo `(id*num)+id-num`.

2.3 *Implementação em Código*

Como linguagem de programação para a codificação do autômato com pilha, pode ser utilizada a linguagem **Python** ou outra linguagem de sua preferência. Se for ulizar **Python** as bibliotecas automata-lib ou automata-python podem ser utilizadas. Lembrando que para o reconhecimento de GLCs, precisamos implementar um **Autômato com Pilha**, com as características da Análise, no caso `SLR(1)`.

A implementação em uma linguagem de programação consiste em simular a execução como foi feito no exemplo da Figura 1. A codificação do comportamento do Autômato com Pilha será dado pela **Tabela de Transição** gerada pelo `JFLAP` ou que pode ser obtida utilizando a ferramenta Grammophone.

Traduzindo a gramática de expressões para o formato da ferramenta Grammophone temos as produções apresentadas no Código 3.

```
S -> E .
S -> V "=" E .
E -> E "+" T .
T -> T "*" F .
F -> "(" E ")" .
```

```
E -> E "-" T .
E -> T .
T -> T "/" F .
T -> F .
F -> V .
V -> id .
F -> num .
```

A ferramenta `Grammophone` irá fazer a análise para alguns tipos de algoritmos de *parsing*. Iremos utilizar a tabela de transições do `SLR(1)` apresentada na Tabela 1.

Tabela 1: Tabela de Transições gerada pelo Gramophone

State	=	+	*****	(	)	-	/	**id**	**num**	$	*S*	*E*	*V*	*T*	*F*
0				shift(7)				shift(5)	shift(8)		1	2	3	4	6
1										accept					
2		shift(9)				shift(10)				reduce(*S E*)					
3	shift(11)	reduce(*F V*)	reduce(*F V*)		reduce(*F V*)	reduce(*F V*)	reduce(*F V*)			reduce(*F V*)					
4		reduce(*E T*)	shift(12)		reduce(*E T*)	reduce(*E T*)	shift(13)			reduce(*E T*)					
5	reduce(*V* **id**)	reduce(*V* **id**)	reduce(*V* **id**)		reduce(*V* **id**)	reduce(*V* **id**)	reduce(*V* **id**)			reduce(*V* **id**)					
6		reduce(*T F*)	reduce(*T F*)		reduce(*T F*)	reduce(*T F*)	reduce(*T F*)			reduce(*T F*)					
7				shift(7)				shift(5)	shift(8)			14	15	4	6
8		reduce(*F* **num**)	reduce(*F* **num**)		reduce(*F* **num**)	reduce(*F* **num**)	reduce(*F* **num**)			reduce(*F* **num**)					
9				shift(7)				shift(5)	shift(8)				15	16	6
10				shift(7)				shift(5)	shift(8)				15	17	6
11				shift(7)				shift(5)	shift(8)			18	15	4	6
12				shift(7)				shift(5)	shift(8)				15		19
13				shift(7)				shift(5)	shift(8)				15		20
14		shift(9)			shift(21)	shift(10)									
15		reduce(*F V*)	reduce(*F V*)		reduce(*F V*)	reduce(*F V*)	reduce(*F V*)			reduce(*F V*)					
16		reduce(*E E* + *T*)	shift(12)		reduce(*E E* + *T*)	reduce(*E E* + *T*)	shift(13)			reduce(*E E* + *T*)					
17		reduce(*E E* - *T*)	shift(12)		reduce(*E E* - *T*)	reduce(*E E* - *T*)	shift(13)			reduce(*E E* - *T*)					
18		shift(9)				shift(10)				reduce(*S V* = *E*)					
19		reduce(*T T* ***** *F*)	reduce(*T T* ***** *F*)		reduce(*T T* ***** *F*)	reduce(*T T* ***** *F*)	reduce(*T T* ***** *F*)			reduce(*T T* ***** *F*)					
20		reduce(*T T* / *F*)	reduce(*T T* / *F*)		reduce(*T T* / *F*)	reduce(*T T* / *F*)	reduce(*T T* / *F*)			reduce(*T T* / *F*)					
21		reduce(*F* (*E*))	reduce(*F* (*E*))		reduce(*F* (*E*))	reduce(*F* (*E*))	reduce(*F* (*E*))			reduce(*F* (*E*))					

Um exemplo de implementação utilizando a biblioteca automata-lib pode ser visto em https://github.com/caleb531/automata/blob/main/tests/test_pda.py.

Um exemplo implementando sem utilizar bibliotecas é apresentado no Código 4.

```python
import sys

def is_accepting(state, accept_states):
    return state in accept_states

def process_input(input_string, states, input_alphabet, stack_alphabet, transitions, start_state, start_symbol, accept_states):
    stack = [start_symbol]
    state = start_state

    print(stack)

    for char in input_string:
        current_symbol = stack[-1]
        print(state, char, current_symbol)
        if (state, char, current_symbol) in transitions:
            new_state, new_symbol = transitions[(state, char, current_symbol)]
            stack.pop()
            print(new_state, new_symbol)

            if new_symbol != 'epsilon':
                stack.extend(list(new_symbol))
```

```
            state = new_state
        else:
            return False

    return is_accepting(state, accept_states) and len(stack) == 1 and stack[0] == start_symbol

if __name__ == "__main__":

    states = {'q0', 'q1'}
    input_alphabet = {'a', 'b'}
    stack_alphabet = {'A', 'B'}
    transitions = {('q0', 'a', 'A'): ('q0', 'AA'),
                   ('q0', 'b', 'A'): ('q1', 'epsilon'),
                   ('q1', 'b', 'A'): ('q1', 'epsilon'),
                   ('q1', 'epsilon', '$'): ('q1', 'epsilon')}
    start_state = 'q0'
    start_symbol = 'A'
    accept_states = {'q1'}

    if len(sys.argv) < 2 :
        raise IOError("Use "+ sys.argv[0] + " file.exp")
    else :
        aux = sys.argv[1].split('.')
        if aux[-1] != 'exp':
            raise IOError("Not a .exp file!")
        data = open(sys.argv[1])

        input_string = data.read()
        input_string = input_string.strip()

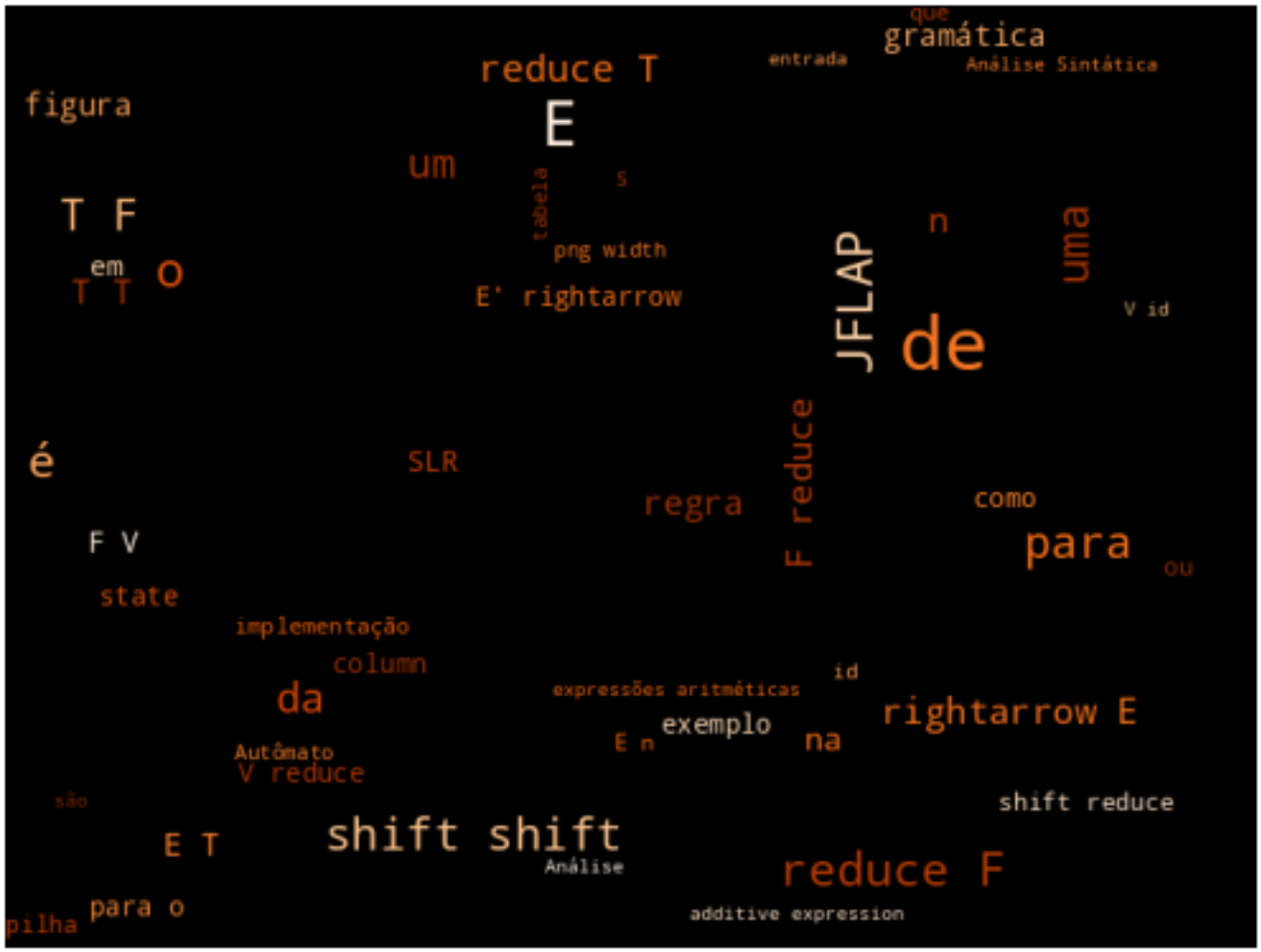
    print('Testing:', input_string)
    if process_input(input_string, states, input_alphabet, stack_alphabet, transitions, start_state, start_symbol, accept_states):
        print("Accepted")
    else:
        print("Rejected")
```

3 Relatório

Escreva um relatório sobre a prática. O relatório deve ter a descrição da atividade e da implementação escolhida (JFLAP ou Python), exemplos de entrada e saída gerada.

4 Referências

4.1 Word Cloud



Referências

AHO, Alfred V., Monica S. LAM, Ravi SETHI, and Jeffrey D. ULLMAN. 2008. *Compiladores: Princípios, Técnicas e Ferramentas*. 2nd ed. São Paulo, SP: Pearson Addison-Wesley.

COOPER, Linda., Keith D.; TORCZON. 2014. *Construindo Compiladores*. 2nd ed. Rio de Janeiro, RJ: Campus.

LOUDEN, Kenneth C. 2004. *Compiladores: Princípios e Práticas*. 1st ed. São Paulo, SP: Thomson.

Menezes, Paulo Blauth. 2011. *Linguagens Formais e Autômatos.* Bookman. https://search.ebscohost.com/login.aspx?direct=true&db=edsmib&AN=edsmib.000000444&lang=pt-br&site=eds-live&scope=site.

Rodger, Susan H. 2006. *JFLAP: An Interactive Formal Languages and Automata Package*. USA: Jones; Bartlett Publishers, Inc. https://www.jflap.org/jflapbook/.

Sipser, Michael. 2007. *Introdução à Teoria Da Computação.* Cengage Learning. https://search.ebscohost.com/login.aspx?direct=true&db=edsmib&AN=edsmib.000008725&lang=pt-br&site=eds-live&scope=site.